

Chapter 2: Core Concepts Commands Reference

Software Images (2.1.2)

Basic Software Image Management

```
Shell
# List all software images
cmsh -c "softwareimage list"

# Show specific software image details
cmsh -c "softwareimage use IMAGE_NAME; show"

# Show default image information
cmsh -c "softwareimage use default-image; show"

# Check software image path and kernel version
cmsh -c "softwareimage use IMAGE_NAME; get path"
cmsh -c "softwareimage use IMAGE_NAME; get kernelversion"
```

Software Image Locking and Synchronization

```
Shell
# Lock a software image (prevents nodes from picking it up)
cmsh -c "softwareimage lock IMAGE_NAME"

# Unlock a software image
cmsh -c "softwareimage unlock IMAGE_NAME"

# Check lock status of images
cmsh -c "softwareimage islocked"

# Force image synchronization without reboot (section 5.6.2)
cmsh -c "device use NODE_NAME; imageupdate"
cmsh -c "device foreach -c CATEGORY (imageupdate)"
```

Software Image Configuration

Shell

```
# Set software image for a node
cmsh -c "device use NODE_NAME; set softwareimage IMAGE_NAME; commit"

# Set software image for a category
cmsh -c "category use CATEGORY_NAME; set softwareimage IMAGE_NAME; commit"

# View which nodes are using an image
cmsh -c "softwareimage use IMAGE_NAME; get nodes"
```

Kernel and Module Management for Images

Shell

```
# View kernel modules for an image
cmsh -c "softwareimage use IMAGE_NAME; kernelmodules; list"

# Add kernel module to an image
cmsh -c "softwareimage use IMAGE_NAME; kernelmodules; add MODULE_NAME"

# Remove kernel module from an image
cmsh -c "softwareimage use IMAGE_NAME; kernelmodules; remove MODULE_NAME"

# Regenerate ramdisk after module changes
cmsh -c "softwareimage use IMAGE_NAME; createramdisk"

# Set kernel parameters for an image
cmsh -c "softwareimage use IMAGE_NAME; set kernelparameters 'PARAM=value'"
cmsh -c "softwareimage use IMAGE_NAME; append kernelparameters '
ADDITIONAL_PARAM=value'"
```

Node Categories (2.1.3)

Category Management

Shell

```
# List all categories
cmsh -c "category list"

# Create a new category
```

```
cmsh -c "category add CATEGORY_NAME"

# Show category configuration
cmsh -c "category use CATEGORY_NAME; show"

# Delete a category
cmsh -c "category remove CATEGORY_NAME"
```

Category Configuration

```
Shell

# Set software image for category
cmsh -c "category use CATEGORY_NAME; set softwareimage IMAGE_NAME; commit"

# Set install mode for category
cmsh -c "category use CATEGORY_NAME; set installmode
[AUTO|FULL|MAIN|NOSYNC|SKIP]; commit"

# Set new node install mode for category
cmsh -c "category use CATEGORY_NAME; set newnodeinstallmode FULL; commit"

# Configure category-level settings
cmsh -c "category use CATEGORY_NAME; set bootloader [grub|syslinux]; commit"
cmsh -c "category use CATEGORY_NAME; set installbootrecord [yes|no]; commit"
```

Node Assignment to Categories

```
Shell

# Assign single node to category
cmsh -c "device use NODE_NAME; set category CATEGORY_NAME; commit"

# Mass assign nodes to category
cmsh -c "device foreach -n node001..node050 (set category CATEGORY_NAME)"
cmsh -c "device commit"

# View nodes in a category
cmsh -c "category use CATEGORY_NAME; get nodes"

# Change default category for new nodes
cmsh -c "partition use base; set defaultcategory CATEGORY_NAME; commit"
```

Category-based Operations

Shell

```
# Operations on entire category
cmsh -c "device foreach -c CATEGORY_NAME (reboot)"
cmsh -c "device foreach -c CATEGORY_NAME (power reset)"
cmsh -c "device foreach -c CATEGORY_NAME (set installmode FULL)"

# Show status of all nodes in category
cmsh -c "device foreach -c CATEGORY_NAME (show status)"
```

Exclude Lists for Categories

Shell

```
# View exclude lists for category
cmsh -c "category use CATEGORY_NAME; get excludelistfullinstall"
cmsh -c "category use CATEGORY_NAME; get excludelistsyncinstall"
cmsh -c "category use CATEGORY_NAME; get excludelistupdate"

# Set exclude lists for category
cmsh -c "category use CATEGORY_NAME; set excludelistfullinstall"
cmsh -c "category use CATEGORY_NAME; set excludelistsyncinstall"
cmsh -c "category use CATEGORY_NAME; set excludelistupdate"
```

Node Groups (2.1.4)

Group Management

Shell

```
# List all node groups
cmsh -c "group list"

# Create a new node group
cmsh -c "group add GROUP_NAME"

# Show group details
cmsh -c "group use GROUP_NAME; show"

# Delete a node group
cmsh -c "group remove GROUP_NAME"
```

Group Membership Management

Shell

Add nodes to a group

```
cmsh -c "group use GROUP_NAME; add nodes NODE_LIST"
```

```
cmsh -c "group use GROUP_NAME; add nodes node001,node002,node003"
```

```
cmsh -c "group use GROUP_NAME; add nodes node001..node010"
```

Remove nodes from a group

```
cmsh -c "group use GROUP_NAME; remove nodes NODE_LIST"
```

View group members

```
cmsh -c "group use GROUP_NAME; get nodes"
```

Add node to multiple groups

```
cmsh -c "device use NODE_NAME; set groups GROUP1,GROUP2,GROUP3; commit"
```

Group-based Operations

Shell

Operations on entire group

```
cmsh -c "device foreach -g GROUP_NAME (reboot)"
```

```
cmsh -c "device foreach -g GROUP_NAME (power reset)"
```

```
cmsh -c "device foreach -g GROUP_NAME (show status)"
```

Set configuration for group (individual nodes)

```
cmsh -c "device foreach -g GROUP_NAME (set installmode FULL)"
```

```
cmsh -c "device commit"
```

Provisioning operations on group

```
cmsh -c "device foreach -g GROUP_NAME (imageupdate)"
```

Special Group Examples

Shell

Create hardware-specific groups

```
cmsh -c "group add brokenhardware"
```

```
cmsh -c "group use brokenhardware; add nodes node087,node783,node917"
```

Create rack-based groups

```
cmsh -c "group add rack5"
```

```
cmsh -c "group use rack5; add nodes node212..node254"
```

```
# Create head node group
cmsh -c "group add headnodes"
cmsh -c "group use headnodes; add nodes mycluster-m1,mycluster-m2"

# Create performance tier groups
cmsh -c "group add top"
cmsh -c "group use top; add nodes node084,node126,node168,node210"
```

Roles (2.1.5)

Role Assignment and Management

```
Shell
# List available roles
cmsh -c "category use CATEGORY_NAME; roles; list"
cmsh -c "device use NODE_NAME; roles; list"

# Assign role to category
cmsh -c "category use CATEGORY_NAME; roles; assign ROLE_NAME"

# Assign role to individual node
cmsh -c "device use NODE_NAME; roles; assign ROLE_NAME"

# Remove role from category
cmsh -c "category use CATEGORY_NAME; roles; unassign ROLE_NAME"

# Remove role from node
cmsh -c "device use NODE_NAME; roles; unassign ROLE_NAME"
```

Common Role Assignments

```
Shell
# Provisioning role assignment
cmsh -c "category use misc; roles; assign provisioning"
cmsh -c "device use node001; roles; assign provisioning"

# Boot role assignment (for DHCP/TFTP scaling)
cmsh -c "device use node001; roles; assign boot"
```

```
# Storage role assignment
cmsh -c "category use storage; roles; assign storage"

# Monitoring role assignment
cmsh -c "category use default; roles; assign monitoring"

# Failover role assignment
cmsh -c "device use head-node-2; roles; assign failover"
```

Role Configuration and Parameters

```
Shell

# Configure provisioning role parameters
cmsh -c "category use misc; roles; use provisioning; set allimages no"
cmsh -c "category use misc; roles; use provisioning; set localimages
default-image"
cmsh -c "category use misc; roles; use provisioning; set provisioningslots 20"
cmsh -c "category use misc; roles; use provisioning; commit"

# Configure provisioning role with node groups
cmsh -c "device use node001; roles; use provisioning; set nodegroups
rack01,rack02"

# Show role configuration
cmsh -c "category use CATEGORY; roles; use ROLE_NAME; show"
cmsh -c "device use NODE_NAME; roles; use ROLE_NAME; show"
```

Role Priority and Overrides

```
Shell

# Role priorities (built-in):
# Category level: 250 (fixed)
# Configuration overlay: 500 (default, variable)
# Node level: 750 (fixed)

# Override category role at node level
cmsh -c "category use compute; roles; assign provisioning"
cmsh -c "category use compute; roles; use provisioning; set provisioningslots
10"
```

```
cmsh -c "device use node001; roles; assign provisioning" # Overrides category
cmsh -c "device use node001; roles; use provisioning; set provisioningslots 5"
# Node-specific
```

Role Import and Export

```
Shell
# Import role configuration from another entity
cmsh -c "category use target; roles; use ROLE_NAME; import source_category"
cmsh -c "device use target_node; roles; use ROLE_NAME; import source_node"

# Copy role configuration between categories
cmsh -c "category use destination; roles; assign ROLE_NAME"
cmsh -c "category use destination; roles; use ROLE_NAME; import
source_category"
```

Chapter 3: Network Configuration Commands Reference

Overview

This chapter provides comprehensive BCM network configuration commands for complex multi-fabric clusters like the demeter DGX H100 BasePOD. Covers management networks, InfiniBand fabrics, BMC interfaces, and VLAN configurations.

Core Network Types in BCM

Default Networks

- **internalnet**: Primary internal cluster network and default management network
- **externalnet**: Network connecting cluster to outside world (corporate/campus network)
- **globalnet**: Special network for domain name resolution (cloud/non-cloud nodes)

Custom Networks for Demeter Cluster

- **Management Network**: In-band management (10.184.164.0/24)
- **BMC Networks**: IPMI-1 (10.184.165.0/25) and IPMI-2 (10.184.165.128/25)

- **InfiniBand Fabrics:** Compute (100.126.0.0/16) and Storage (100.127.0.0/16)

Detailed Network Configuration Commands

Basic Network Management

Network Creation and Configuration

Shell

Create new network

```
cmsh -c "network add NETWORK_NAME"
```

Configure network parameters

```
cmsh -c "network use NETWORK_NAME; set baseaddress IP_ADDRESS"
```

```
cmsh -c "network use NETWORK_NAME; set netmaskbits BITS"
```

```
cmsh -c "network use NETWORK_NAME; set gateway GATEWAY_IP"
```

```
cmsh -c "network use NETWORK_NAME; set domainname DOMAIN_NAME"
```

```
cmsh -c "network use NETWORK_NAME; set type [internal|external]"
```

```
cmsh -c "network use NETWORK_NAME; set mtu MTU_SIZE"
```

```
cmsh -c "network use NETWORK_NAME; commit"
```

Network booting and DHCP settings

```
cmsh -c "network use NETWORK_NAME; set nodebooting yes"
```

```
cmsh -c "network use NETWORK_NAME; set lockdownhcpd yes"
```

```
cmsh -c "network use NETWORK_NAME; set dynamicrangestart START_IP"
```

```
cmsh -c "network use NETWORK_NAME; set dynamicrangeend END_IP"
```

Interface Management

Shell

Add interface to single node

```
cmsh -c "device use NODE_NAME; interfaces; add TYPE INTERFACE_NAME"
```

```
cmsh -c "device use NODE_NAME; interfaces use INTERFACE_NAME; set network  
NETWORK_NAME"
```

```
cmsh -c "device use NODE_NAME; interfaces use INTERFACE_NAME; set ip  
IP_ADDRESS"
```

```
cmsh -c "device use NODE_NAME; interfaces use INTERFACE_NAME; commit"
```

Mass interface addition

```
cmsh -c "device addinterface -n node001..node100 TYPE INTERFACE_NAME  
NETWORK_NAME FIRST_IP"
```

```
cmsh -c "device commit"
```

```
# Interface types: physical, vlan, bond, bridge, bmc, alias, tunnel
```

VLAN Configuration

Basic VLAN Setup

Shell

```
# Add VLAN interface
```

```
cmsh -c "device use NODE_NAME; interfaces; add vlan INTERFACE.VLAN_ID"
cmsh -c "device use NODE_NAME; interfaces use INTERFACE.VLAN_ID; set network NETWORK_NAME"
cmsh -c "device use NODE_NAME; interfaces use INTERFACE.VLAN_ID; set ip IP_ADDRESS"
cmsh -c "device use NODE_NAME; interfaces use INTERFACE.VLAN_ID; commit"
```

```
# Configure VLAN header reordering
```

```
cmsh -c "device use NODE_NAME; interfaces use INTERFACE.VLAN_ID; set reorderhdr yes"
```

VLAN Provisioning Support

Shell

```
# Add 8021q module to software image for VLAN support
```

```
cmsh -c "softwareimage use default-image; kernelmodules; add 8021q; commit"
```

```
# Set VLAN ID in kernel parameters
```

```
cmsh -c "softwareimage use default-image; append kernelparameters ' VLANID=89'; commit"
```

InfiniBand Configuration

InfiniBand Network Setup

Shell

```
# Create InfiniBand network
```

```
cmsh -c "network add ibnet"
cmsh -c "network use ibnet; set baseaddress 10.149.0.0"
cmsh -c "network use ibnet; set netmaskbits 16"
```

```
cmsh -c "network use ibnet; set domainname ib.cluster"
cmsh -c "network use ibnet; set type internal"
cmsh -c "network use ibnet; set mtu 4096; commit" # datagram mode
# cmsh -c "network use ibnet; set mtu 65536; commit" # connected mode

# Enable network booting over InfiniBand
cmsh -c "network use ibnet; set nodebooting yes; commit"
```

InfiniBand Interface Configuration

```
Shell
# Add InfiniBand interface to head node
cmsh -c "device use HEAD_NODE; interfaces; add physical ib0"
cmsh -c "device use HEAD_NODE; interfaces use ib0; set network ibnet"
cmsh -c "device use HEAD_NODE; interfaces use ib0; set ip 10.149.255.254;
commit"

# Mass add InfiniBand interfaces to nodes
cmsh -c "device addinterface -n node001..node150 physical ib0 ibnet 10.149.0.1"
cmsh -c "device commit"

# Configure connected mode (if needed)
cmsh -c "device use NODE_NAME; interfaces use ib0; set connectedmode yes;
commit"

# Set IPoIB mode for provisioning
echo datagram > /cm/node-installer/scripts/ipoib_mode
# echo connected > /cm/node-installer/scripts/ipoib_mode
```

Subnet Manager Configuration

```
Shell
# Assign subnet manager role
cmsh -c "device use node001; roles; assign subnetmanager; commit"

# Alternative: Configure as service
cmsh -c "device use node001; services; add opensm"
cmsh -c "device use node001; services use opensm; set autostart yes"
cmsh -c "device use node001; services use opensm; set monitored yes; commit"
```

InfiniBand Verification

```
Shell
# Check InfiniBand status
ibstat

# Test connectivity
ping node015.ib.cluster

# Performance testing with Intel MPI Benchmark
cd /cm/shared/apps/imb/current/
./setup.sh
cd ~/BenchMarks/imb/2017
module load openmpi/gcc
make -f make_mpi2
mpirun -np 2 -machinefile ../nodes IMB-MPI1 PingPong
```

BMC Interface Configuration

BMC Network Setup

```
Shell
# Create BMC network
cmsh -c "network add bmcnet"
cmsh -c "network use bmcnet; set baseaddress 10.148.0.0"
cmsh -c "network use bmcnet; set netmaskbits 16"
cmsh -c "network use bmcnet; set domainname bmc.cluster"
cmsh -c "network use bmcnet; set type internal; commit"
```

BMC Interface Assignment

```
Shell
# Add BMC interface to single node
cmsh -c "device use NODE_NAME; interfaces; add bmc ipmi0"
cmsh -c "device use NODE_NAME; interfaces use ipmi0; set network bmcnet"
cmsh -c "device use NODE_NAME; interfaces use ipmi0; set ip IP_ADDRESS; commit"

# Mass add BMC interfaces
cmsh -c "device addinterface -n node001..node150 bmc ipmi0 bmcnet 10.148.0.1"
cmsh -c "device commit"

# Add BMC interface to head node (alias interface for same physical network)
```

```
cmsh -c "device use HEAD_NODE; interfaces; add alias eth0:0"
cmsh -c "device use HEAD_NODE; interfaces use eth0:0; set network bmcnet"
cmsh -c "device use HEAD_NODE; interfaces use eth0:0; set ip 10.148.255.254;
commit"
```

BMC Authentication Configuration

Shell

Set global BMC credentials

```
cmsh -c "partition use base; bmcsettings"
cmsh -c "partition use base; bmcsettings; set username bmcadmin"
cmsh -c "partition use base; bmcsettings; set password" # Will prompt
cmsh -c "partition use base; bmcsettings; set userid 4" # Administrator access
cmsh -c "partition use base; bmcsettings; commit"
```

Category-level BMC settings

```
cmsh -c "category use dgx-nodes; bmcsettings; set username dgxadmin; commit"
```

Node-level BMC settings (override category/global)

```
cmsh -c "device use node001; bmcsettings; set username nodeadmin; commit"
```

BMC Interface Types and Naming

Shell

Recommended interface naming conventions:

IPMI: ipmi0, ipmi1, etc.

iLO: ilo0, ilo1, etc.

DRAC: drac0, drac1, etc.

CIMC: cimc0, cimc1, etc.

Redfish: rf0, rf1, etc.

Static vs DHCP IP assignment

```
cmsh -c "device use NODE_NAME; interfaces use ipmi0; set dhcp yes" # DHCP (not
recommended for power management)
```

```
cmsh -c "device use NODE_NAME; interfaces use ipmi0; set ip STATIC_IP" #
Static (recommended)
```

Advanced Network Features

Bonded Interfaces

Shell

Create bonded interface

```
cmsh -c "device use NODE_NAME; interfaces; add bond bond0"
cmsh -c "device use NODE_NAME; interfaces use bond0; set slaves eth0,eth1"
cmsh -c "device use NODE_NAME; interfaces use bond0; set network NETWORK_NAME"
cmsh -c "device use NODE_NAME; interfaces use bond0; set ip IP_ADDRESS; commit"
```

Set bonding mode and options

```
cmsh -c "device use NODE_NAME; interfaces use bond0; set bondmode
active-backup"
cmsh -c "device use NODE_NAME; interfaces use bond0; set bondoptions
'miimon=100'"
```

Bridge Interfaces

Shell

Create bridge interface

```
cmsh -c "device use NODE_NAME; interfaces; add bridge br0"
cmsh -c "device use NODE_NAME; interfaces use br0; set slaves eth0,eth1"
cmsh -c "device use NODE_NAME; interfaces use br0; set network NETWORK_NAME"
cmsh -c "device use NODE_NAME; interfaces use br0; set ip IP_ADDRESS; commit"
```

Static Routes

Shell

Add static route

```
cmsh -c "device use NODE_NAME; staticroutes; add ROUTE_NAME"
cmsh -c "device use NODE_NAME; staticroutes use ROUTE_NAME; set destination
DEST_IP"
cmsh -c "device use NODE_NAME; staticroutes use ROUTE_NAME; set gateway
GATEWAY_IP"
cmsh -c "device use NODE_NAME; staticroutes use ROUTE_NAME; set
networkdevicename INTERFACE"
cmsh -c "device use NODE_NAME; staticroutes use ROUTE_NAME; commit"
```

Category-level static routes

```
cmsh -c "category use CATEGORY; staticroutes; add ROUTE_NAME"
```

```
cmsh -c "category use CATEGORY; staticroutes use ROUTE_NAME; set destination  
DEST_IP"  
cmsh -c "category use CATEGORY; staticroutes use ROUTE_NAME; set gateway  
GATEWAY_IP; commit"
```

Additional Hostnames and DNS

```
Shell  
  
# Add additional hostnames to interface  
cmsh -c "device use NODE_NAME; interfaces use INTERFACE; set  
additionalhostnames 'extra01 extra02'"  
cmsh -c "device use NODE_NAME; interfaces use INTERFACE; commit"  
  
# Add hostnames to head node for internal DNS  
cmsh -c "device use HEAD_NODE; interfaces use eth1; set additionalhostnames  
'test special'"  
cmsh -c "device use HEAD_NODE; interfaces use eth1; commit"  
  
# Set DNS resolution priority  
cmsh -c "device use NODE_NAME; interfaces use INTERFACE; set onnetworkpriority  
100"
```

Network Troubleshooting Commands

Connectivity Testing

```
Shell  
  
# Basic connectivity testing  
cmsh -c "device connectivity"  
cmsh -c "device connectivity --statistics --count 100 --delay 0.01"  
cmsh -c "device connectivity --network NETWORK_NAME"  
  
# Test specific network performance  
/cm/local/apps/cmd/scripts/cm-iperf.py -n node001..node010  
/cm/local/apps/cmd/scripts/cm-iperf.py -n node001..node010 --count 10 -r -p 2  
  
# BMC connectivity testing  
for i in $(cmsh -c "device; foreach -t physicalnode (interfaces; use ipmi0; get  
ip)"); do  
    ping -c1 $i
```

```
done | grep -B1 packet
```

Network Analysis

Shell

View all routes

```
cmsh -c "device routes"
```

```
cmsh -c "device routes --category CATEGORY | awk 'if ($3 != \"EXPECTED_GATEWAY\")  
print $0'"
```

Check active connections

```
cmsh -c "device connections"
```

```
cmsh -c "device connections | grep Listen | grep ' 53'" # DNS services
```

```
cmsh -c "device connections | awk 'print $4' | sort -un" # All used ports
```

Network interface status

```
cmsh -c "device foreach (interfaces; list)"
```

```
cmsh -c "device foreach -n node001..node010 (interfaces; list)" | grep DOWN
```

Configuration Validation

Shell

Validate network configuration

```
cmsh -c "network list"
```

```
cmsh -c "network use NETWORK_NAME; show"
```

Check interface assignments

```
cmsh -c "device use NODE_NAME; interfaces; list"
```

```
cmsh -c "device foreach -c CATEGORY (interfaces; list)" | grep NETWORK_NAME
```

Verify DNS resolution

```
cmsh -c "device foreach (!nslookup HOSTNAME)"
```


Demeter Cluster Specific Commands

Complete Demeter Network Setup

Shell

1. Management Network Setup

```
cmsh -c "network add managementnet"
cmsh -c "network use managementnet; set baseaddress 10.184.164.0; set
netmaskbits 24"
cmsh -c "network use managementnet; set gateway 10.184.164.1; set domainname
demeter.local"
cmsh -c "network use managementnet; set type internal; commit"
```

2. IPMI Networks Setup

```
cmsh -c "network add oobmanagementnet"
cmsh -c "network use oobmanagementnet; set baseaddress 10.184.165.0; set
netmaskbits 25"
cmsh -c "network use oobmanagementnet; set gateway 10.184.165.1; commit"

cmsh -c "network add oobmanagementnet2"
cmsh -c "network use oobmanagementnet2; set baseaddress 10.184.165.128; set
netmaskbits 25"
cmsh -c "network use oobmanagementnet2; set gateway 10.184.165.129; commit"
```

3. InfiniBand Networks Setup

```
cmsh -c "network add computenet"
cmsh -c "network use computenet; set baseaddress 100.126.0.0; set netmaskbits
16"
cmsh -c "network use computenet; set domainname compute.demeter.local; set mtu
4096; commit"

cmsh -c "network add storagenet"
cmsh -c "network use storagenet; set baseaddress 100.127.0.0; set netmaskbits
16"
cmsh -c "network use storagenet; set domainname storage.demeter.local; set mtu
4096; commit"
```

4. Management Node Interfaces

```
cmsh -c "device use demeter-mgmt-1; interfaces; add physical eth0"
cmsh -c "device use demeter-mgmt-1; interfaces use eth0; set network
managementnet"
cmsh -c "device use demeter-mgmt-1; interfaces use eth0; set ip 10.184.164.51;
commit"
```

5. DGX Node Interfaces (example for dgx-01)

```

cmsh -c "device use dgx-01; interfaces; add physical eth0"
cmsh -c "device use dgx-01; interfaces use eth0; set network managementnet"
cmsh -c "device use dgx-01; interfaces use eth0; set ip 10.184.164.10; commit"

cmsh -c "device use dgx-01; interfaces; add bmc ipmi0"
cmsh -c "device use dgx-01; interfaces use ipmi0; set network
oobmanagementnet2"
cmsh -c "device use dgx-01; interfaces use ipmi0; set ip 10.184.165.130;
commit"

```

Mass Configuration for All DGX Nodes

```

Shell
# Configure all 31 DGX nodes at once
cmsh -c "device addinterface -n dgx-01..dgx-31 physical eth0 managementnet
10.184.164.10"
cmsh -c "device addinterface -n dgx-01..dgx-31 bmc ipmi0 oobmanagementnet2
10.184.165.130"
cmsh -c "device addinterface -n dgx-01..dgx-31 physical ib0 computenet
100.126.0.1"
cmsh -c "device addinterface -n dgx-01..dgx-31 physical ib1 storagenet
100.127.0.1"
cmsh -c "device commit"

```

Quick Reference Patterns

Network Creation Pattern

```

Shell
cmsh -c "network add NAME; set baseaddress IP; set netmaskbits BITS; set
gateway GW; commit"

```

Interface Addition Pattern

```

Shell
cmsh -c "device use NODE; interfaces; add TYPE NAME; set network NET; set ip
IP; commit"

```

Mass Interface Pattern

Shell

```
cmsh -c "device addinterface -n node001..nodeN TYPE NAME NETWORK FIRST_IP;
commit"
```

BMC Setup Pattern

Shell

```
cmsh -c "device addinterface -n node001..nodeN bmc ipmi0 bmcnet FIRST_IP;
commit"
cmsh -c "partition use base; bmcsettings; set username USER; set password PASS;
commit"
```

Chapter 4: Power Management Commands Reference

Overview

Power management in BCM includes controlling main power supply through PDUs, BMCs, monitoring power consumption, and ensuring safe failover operations. Critical for DGX H100 node resets and firmware updates in the demeter cluster.

Power Control Methods

PDU-Based Power Control

Shell

Add PDU device to cluster

```
cmsh -c "device add mypdu type PowerDistributionUnit"
cmsh -c "device use mypdu; set hostname PDU_IP_ADDRESS; commit"
```

Configure node to use PDU ports (multiple ports for redundancy)

```
cmsh -c "device use node001; set powerdistributionunits mypdu:2 mypdu:4;
commit"
```

Configure APC PDU power control

```
cmsh -c "device use node001; set powercontrol apc; commit"

# Show PDU configuration
cmsh -c "device use node001; show | grep -i power"
cmsh -c "device use node001; get powerdistributionunits"

# Remove PDU port from node
cmsh -c "device use node001; removefrom powerdistributionunits mypdu:7"
```

IPMI/BMC-Based Power Control

```
Shell

# Configure IPMI power control for single node
cmsh -c "device use node001; set powercontrol ipmi0; commit"

# Configure IPMI power control for all nodes in category
cmsh -c "device foreach -c default (set powercontrol ipmi0; commit)"

# Configure for DGX nodes with BMC interfaces
cmsh -c "device foreach -c dgx-h100 (set powercontrol ipmi0; commit)"

# Check IPMI power configuration
cmsh -c "device use node001; show | grep -i power"
cmsh -c "device use node001; power status"
```

Vendor-Specific Power Control

```
Shell

# HP iLO configuration
cmsh -c "device foreach -c hpe-nodes (set powercontrol ilo0; commit)"

# Dell DRAC configuration
cmsh -c "device foreach -c dell-nodes (set powercontrol drac0; commit)"

# Cisco CIMC configuration
cmsh -c "device foreach -c cisco-nodes (set powercontrol cimc0; commit)"

# Redfish-based power control (modern standard)
cmsh -c "device foreach -c redfish-nodes (set powercontrol redfish0; commit)"
```

Custom Power Control Scripts

Shell

Set custom power script

```
cmsh -c "device use node001; set powercontrol custom; commit"
```

```
cmsh -c "device use node001; set custompowerscript /path/to/script.sh; commit"
```

Set custom script argument (e.g., MAC address for Wake-on-LAN)

```
cmsh -c "device use node001; set custompowerscriptargument MAC_ADDRESS; commit"
```

Example script template locations

```
ls /cm/local/examples/cmd/custompower
```

Power Operations Commands

Basic Power Operations

Shell

Power on single node

```
cmsh -c "device power -n node001 on"
```

Power off single node

```
cmsh -c "device power -n node001 off"
```

Power reset single node (hard power cycle)

```
cmsh -c "device power -n node001 reset"
```

Check power status

```
cmsh -c "device power -n node001 status"
```

```
cmsh -c "device power status -c dgx-h100"
```

Multi-Node Power Operations

Shell

Power on range of nodes

```
cmsh -c "device power -n node001..node010 on"
```

```
cmsh -c "device power -n node001,node005,node010 on"
```

Power operations by category (critical for DGX management)

```
cmsh -c "device power on -c dgx-h100"
```

```
cmsh -c "device power off -c dgx-h100"
```

```
cmsh -c "device power status -c dgx-h100"

# Power operations by group
cmsh -c "device power on -g compute-nodes"
cmsh -c "device power off -g storage-nodes"

# Power operations by rack (demeter cluster rack management)
cmsh -c "device power on -r rack01"
cmsh -c "device power off -r rack01..rack04"
```

Power Surge Management and Delays

```
Shell

# Power on with custom delay between nodes (default 1s)
cmsh -c "device power -n node001..node020 -d 2.0 on"
cmsh -c "device power -n node001..node020 -d 0.1 on" # 100ms delay

# Power up in batches (critical for large clusters)
cmsh -c "device power on -p 3 rack[01-12]" # 3 racks at a time
cmsh -c "device power on -p 4 -n node001..node080" # 4 nodes at a time

# Custom parallel delay between batches (default 20s)
cmsh -c "device power on -p 4 --parallel-delay 30 rack[01-20]"

# Zero delay (use with caution - power surge risk)
cmsh -c "device power -n node001..node010 -d 0 on"
```

Scheduled Power Operations

```
Shell

# Schedule power operation at specific time
cmsh -c "device power off --at 23:55 -c maintenance"
cmsh -c "device power on --at 08:00 -c dgx-h100"

# Schedule power operation after delay
cmsh -c "device power off --after 600 -n node001" # 10 minutes
cmsh -c "device power reset --after 1h -n node001" # 1 hour

# Scheduled batch operations with parallel delay
cmsh -c "device power on -p 2 --parallel-delay 45 --at 06:00 rack[01-10]"
```

```
# Dry run to see scheduled times
cmsh -c "device power on -p 4 --parallel-dry-run rack[01-20]"
```

Power Operation Management

```
Shell

# List pending power operations
cmsh -c "device power list"

# Show operations waiting to execute
cmsh -c "device power wait"

# Wait for specific operation to complete
cmsh -c "device power wait 1"
cmsh -c "device power wait all"
cmsh -c "device power wait last"

# Cancel pending power operations
cmsh -c "device power cancel node001"
cmsh -c "device power cancel -c dgx-h100"
```

Power History and Monitoring

```
Shell

# Show power operation history (last 8 operations)
cmsh -c "device power history"
cmsh -c "device power history -n node001"
cmsh -c "device power history -c dgx-h100"

# Power status with overview
cmsh -c "device power status -w -c dgx-h100"

# Force power operations on closed nodes
cmsh -c "device power -f reset -n node001"
```

Advanced Power Operations

Shell

Retry failed power operations

```
cmsh -c "device power on --retry-count 3 --retry-delay 5 -n node001"
```

Background power operations (returns immediately)

```
cmsh -c "device power -b on -c dgx-h100"
```

Direct PDU port operations

```
cmsh -c "device power on --port pdu1:1"
```

```
cmsh -c "device power off --port pdu1:[1-4]"
```

Filter by node status

```
cmsh -c "device power status -s UP -c dgx-h100"
```

```
cmsh -c "device power on -s DOWN -c dgx-h100"
```

Power Monitoring Commands

PDU Monitoring Setup

Shell

Monitor PDU metrics (requires monitoring system)

PDUBankLoad: Phase load in amperes for specific bank

PDULoad: Total phase load in amperes for PDU

Check PDU status and load

```
cmsh -c "device use pdu01; show"
```

```
cmsh -c "monitoring metric show PDULoad"
```

```
cmsh -c "monitoring metric show PDUBankLoad"
```

DGX H100 Specific Power Management

DGX Power Considerations

Shell

DGX H100 requires careful power management due to:

- High power consumption (10.2kW max per node)

- AC power cycling required for some firmware updates

- Multiple power feeds for redundancy


```
# DGX power operations with extended delays
cmsh -c "device power -n dgx001..dgx008 -d 5.0 on" # 5 second delay
cmsh -c "device power reset --after 30 -n dgx001" # 30 second delay for reset

# Batch power operations for DGX racks
cmsh -c "device power on -p 2 --parallel-delay 60 rack[01-04]" # 2 DGX at a
time
```

Demeter Cluster Power Management

```
Shell
# Based on demeter-desired-state.yaml structure
# Management network: 10.184.164.0/24
# IPMI-1: 10.184.165.0/25
# IPMI-2: 10.184.165.128/25

# Configure demeter DGX nodes for IPMI power control
cmsh -c "device foreach -c dgx-h100 (set powercontrol ipmi0; commit)"

# Power operations for demeter compute nodes
cmsh -c "device power status -g compute-fabric"
cmsh -c "device power on -g storage-fabric -d 3.0"

# Staggered power-up for demeter cluster
cmsh -c "device power on -p 1 --parallel-delay 120 -c dgx-h100" # 1 DGX every
2 minutes
```

Combined PDU and IPMI Power Control

PowerOffPDUOutlet Configuration

```
Shell
# Enable PDU outlet power-off after IPMI shutdown (saves additional watts)
# Edit /cm/local/apps/cmd/etc/cmd.conf
# Set: PowerOffPDUOutlet = true
# Restart CMDaemon: systemctl restart cmd

# This configuration:
```

- # 1. Sends IPMI power off command
- # 2. Subsequently powers off PDU port
- # 3. Shuts down BMC to save power
- # 4. Requires BIOS auto-power-on when AC restored

Emergency Power Operations

Emergency Shutdown Procedures

Shell

```
# Emergency shutdown all compute nodes
cmsh -c "device power off -c dgx-h100 -d 0.5"

# Emergency shutdown by rack (staggered)
cmsh -c "device power off -r rack01..rack04 -p 1 --parallel-delay 10"

# Force shutdown unresponsive nodes
cmsh -c "device power -f off -s 'CLOSED|DOWN' -c dgx-h100"
```

Recovery Procedures

Shell

```
# Staged recovery power-up
cmsh -c "device power on -p 1 --parallel-delay 300 -c dgx-h100" # 5-minute intervals

# Check power status during recovery
watch -n 10 'cmsh -c "device power status -c dgx-h100"'

# Verify all nodes powered up successfully
cmsh -c "device power status -c dgx-h100" | grep -v "ON" || echo "All nodes online"
```

Best Practices for DGX Infrastructure

Pre-Firmware Update Power Sequence

Shell

```
# 1. Verify current power state
cmsh -c "device power status -n dgx001"

# 2. Graceful shutdown (if OS responsive)
cmsh -c "device power off -n dgx001"

# 3. Wait for complete shutdown (30 seconds minimum)
sleep 30

# 4. Verify OFF state
cmsh -c "device power status -n dgx001"

# 5. Proceed with firmware update
# 6. AC power cycle after firmware update
cmsh -c "device power reset -n dgx001"
```

Post-Reset Validation

Shell

```
# Verify power restoration
cmsh -c "device power status -n dgx001"

# Check BMC accessibility
cmsh -c "device use dgx001; bmc; show"

# Validate network connectivity
cmsh -c "device use dgx001; interfaces; list"
```

Chapter 5: Node Provisioning Commands Reference

Core BCM Administrative Commands

Node Management

Shell

Basic node operations

```
cmsh -c "device use NODE_NAME; show"
cmsh -c "device use NODE_NAME; show status"
cmsh -c "device use NODE_NAME; reboot"
cmsh -c "device use NODE_NAME; power [on|off|reset]"
```

Node configuration

```
cmsh -c "device use NODE_NAME; set mac MAC_ADDRESS"
cmsh -c "device use NODE_NAME; set softwareimage IMAGE_NAME"
cmsh -c "device use NODE_NAME; set category CATEGORY_NAME"
cmsh -c "device use NODE_NAME; commit"
```

Install Mode Management

Shell

Install mode settings

```
cmsh -c "device use NODE_NAME; set installmode [AUTO|FULL|MAIN|NOSYNC|SKIP]"
cmsh -c "device use NODE_NAME; set nextinstallmode
[AUTO|FULL|MAIN|NOSYNC|SKIP]"
cmsh -c "device use NODE_NAME; set pxelabel LABEL_NAME"
cmsh -c "category use CATEGORY; set newnodeinstallmode FULL"
```

Data node protection

```
cmsh -c "device use NODE_NAME; set datanode yes"
```

PXE and Boot Configuration

Shell

PXE label management

```
cmsh -c "device use NODE_NAME; set pxelabel MEMTEST"
cmsh -c "device use NODE_NAME; clear pxelabel"
cmsh -c "device foreach -c default (set pxelabel MEMTEST)"
```

Boot configuration

```
cmsh -c "device use NODE_NAME; set bootloader [grub|syslinux]"
cmsh -c "device use NODE_NAME; set installbootrecord yes"
cmsh -c "network use NETWORK_NAME; set nodebooting yes"
```

Provisioning Management

Shell

Provisioning status and control

```
cmsh -c "softwareimage provisioningstatus"
cmsh -c "softwareimage provisioningstatus -r" # Show requests
cmsh -c "softwareimage provisioningstatus -a" # Show all details
```

Provisioning node management

```
cmsh -c "softwareimage updateprovisioners"
cmsh -c "softwareimage updateprovisioners IMAGE_NAME"
cmsh -c "softwareimage drain -n NODE_NAME"
cmsh -c "softwareimage undrain -n NODE_NAME"
cmsh -c "softwareimage drain --role provisioning"
```

Image locking

```
cmsh -c "softwareimage lock IMAGE_NAME"
cmsh -c "softwareimage unlock IMAGE_NAME"
cmsh -c "softwareimage islocked"
```

Network Configuration

Shell

Network interface management

```
cmsh -c "device use NODE_NAME; interfaces; list"
cmsh -c "device use NODE_NAME; interfaces; set INTERFACE ip IP_ADDRESS"
cmsh -c "device use NODE_NAME; interfaces; set INTERFACE network NETWORK_NAME"
cmsh -c "device use NODE_NAME; interfaces; set INTERFACE networkdevicename eth0"
```

Provisioning interface

```
cmsh -c "device use NODE_NAME; set provisioninginterface INTERFACE_NAME"
cmsh -c "device use NODE_NAME; set provisioningtransport [rsyncdaemon|rsyncssh]"
```

Network protocols

```
cmsh -c "device use NODE_NAME; set bootloaderprotocol [HTTP|HTTPS|TFTP]"
```

BMC Management

Shell

BMC configuration

```
cmsh -c "device use NODE_NAME; bmc; set ipaddress IP_ADDRESS"
```

```
cmsh -c "device use NODE_NAME; bmc; set username USERNAME"
```

```
cmsh -c "device use NODE_NAME; bmc; set password PASSWORD"
```

```
cmsh -c "device use NODE_NAME; bmc; show"
```

```
cmsh -c "device use NODE_NAME; bmc; commit"
```

BMC network configuration

```
cmsh -c "device use NODE_NAME; bmc; set netmask NETMASK"
```

```
cmsh -c "device use NODE_NAME; bmc; set gateway GATEWAY_IP"
```

Certificate Management

Shell

Certificate operations

```
cmsh -c "cert listrequests"
```

```
cmsh -c "cert issuecertificate REQUEST_ID"
```

Auto-signing configuration

```
cmsh -c "partition use base; set signinstallercertificates [AUTO|MANUAL]"
```

```
cmsh -c "network use NETWORK_NAME; set allowautosign  
[always|automatic|never|secret]"
```

Software Image Management

Shell

Image operations

```
cmsh -c "softwareimage list"
```

```
cmsh -c "softwareimage use IMAGE_NAME; show"
```

```
cmsh -c "softwareimage use IMAGE_NAME; kernelmodules; list"
```

```
cmsh -c "softwareimage use IMAGE_NAME; kernelmodules; add MODULE_NAME"
```

```
cmsh -c "softwareimage use IMAGE_NAME; createramdisk"
```

Exclude lists

```
cmsh -c "category use CATEGORY; get excludelistfullinstall"
cmsh -c "category use CATEGORY; set excludelistfullinstall"
cmsh -c "category use CATEGORY; get excludelistsyncinstall"
```

Node Discovery and Assignment

Shell

New node management

```
cmsh -c "device newnodes"
cmsh -c "device newnodes -w -n node001..node008"
cmsh -c "device newnodes -s -n node001..node008" # Save (commit)
cmsh -c "device newnodes -f -n NODE_NAME" # Force assignment
cmsh -c "device newnodes -o OFFSET -n node001..node008" # Skip nodes
```

Grouping assignments

```
cmsh -c "device newnodes -s -r rack01..rack03"
cmsh -c "device newnodes -s -c CATEGORY_NAME"
cmsh -c "device newnodes -s -g GROUP_NAME"
```

Switch port management

```
cmsh -c "device showport MAC_ADDRESS"
cmsh -c "device use NODE_NAME; set switchports SWITCH:PORT"
cmsh -c "device use NODE_NAME; clear switchports"
```

Troubleshooting Commands

Shell

Sync and provisioning logs

```
cmsh -c "device use NODE_NAME; synclog"
cmsh -c "device use NODE_NAME; synclog -p" # Show log path
cmsh -c "device use NODE_NAME; syncinfo"
```

Installer interactions

```
cmsh -c "device installerinteractions -w -n NODE_NAME --confirm"
cmsh -c "device installerinteractions -w -n NODE_NAME --deny"
```

Provisioning request management

```
cmsh -c "softwareimage cancelprovisioningrequest -a"
```

```
cmsh -c "softwareimage cancelprovisioningrequest REQUEST_ID"
```

Mass Operations

Shell

Bulk node operations

```
cmsh -c "device foreach -n node001..node100 (COMMAND)"
cmsh -c "device foreach -c CATEGORY (COMMAND)"
cmsh -c "device foreach -r RACK_NAME (COMMAND)"
cmsh -c "device foreach -g GROUP_NAME (COMMAND)"
```

Examples

```
cmsh -c "device foreach -c default (set installmode FULL)"
cmsh -c "device foreach -n node001..node010 (reboot)"
cmsh -c "device foreach -r rack01 (power reset)"
```

Advanced Configuration

Shell

Kernel modules and parameters

```
cmsh -c "softwareimage use IMAGE_NAME; kernelmodules; add MODULE_NAME"
cmsh -c "softwareimage use IMAGE_NAME; append kernelparameters '
PARAMETER=value'"
```

VLAN configuration

```
cmsh -c "softwareimage use IMAGE_NAME; kernelmodules; add 8021q"
cmsh -c "softwareimage use IMAGE_NAME; append kernelparameters ' VLANID=89'"
```

InfiniBand setup

```
cmsh -c "network use ibnet; set nodebooting yes"
cmsh -c "device use NODE_NAME; set provisioninginterface ib0"
```

Filesystem and Storage

Shell

Fspart management

```
cmsh -c "fspart list"
cmsh -c "fspart info"
```



```
cmsh -c "fspart info -s" # With size information
cmsh -c "fspart trigger --all"
cmsh -c "fspart lock /path/to/fspart"
cmsh -c "fspart unlock /path/to/fspart"
cmsh -c "fspart locked"
```

Quick Reference One-Liners

Common Operations

```
Shell
# Full node reset
cmsh -c "device use node001; set installmode FULL; commit; reboot"

# Check all node status
cmsh -c "device foreach (show status)"

# Mass power cycle
cmsh -c "device foreach -n node001..node010 (power reset)"

# Emergency stop all provisioning
cmsh -c "softwareimage cancelprovisioningrequest -a"

# Lock image during maintenance
cmsh -c "softwareimage lock default-image"

# Force new node assignment
cmsh -c "device newnodes -f -n node001"
```

Emergency Recovery

```
Shell
# Node stuck in installer
cmsh -c "device installerinteractions -w -n node001 --deny"

# Clear bad MAC assignment
cmsh -c "device use node001; clear mac; commit"

# Reset to maintenance mode
```

```
cmsh -c "device use node001; set installmode MAIN; commit; reboot"

# Skip sync for emergency boot
cmsh -c "device use node001; set nextinstallmode NOSYNC; commit; reboot"
```

Chapter 16: BIOS and Firmware Management Commands Reference

Overview

Modern BCM BIOS and firmware management uses the Redfish standard for RESTful management of large numbers of nodes. This chapter covers commands for BIOS configuration, firmware updates, and hardware profile verification.

BIOS Management via Redfish

BIOS Model Configuration

```
Shell

# Set BIOS model for a node (required first step)
cmsh -c "device use node001; biossettings; set model<TAB><TAB>"
cmsh -c "device use node001; biossettings; set model dell_r730; commit"
cmsh -c "device use node001; biossettings; set model hpe_dl380; commit"
cmsh -c "device use node001; biossettings; set model dell_15g-intel; commit"

# Set BIOS model for a category
cmsh -c "category use dgx-h100; biossettings; set model dell_15g-intel; commit"
```

BIOS Configuration Status and Viewing

```
Shell

# View BIOS configuration status (Configured/Pending/Live states)
cmsh -c "device use node001; biossettings; status"

# Check BIOS differences between configured and detected
```

```
cmsh -c "device use node001; bios check"
```

```
# View specific BIOS parameter
```

```
cmsh -c "device use node001; biossettings; status | grep 'Boot Mode'"
```

```
cmsh -c "device use node001; biossettings; status | grep 'Node Interleaving'"
```

BIOS Parameter Configuration

Shell

```
# Configure Boot Mode
```

```
cmsh -c "device use node001; biossettings; set boot mode<TAB><TAB>"
```

```
cmsh -c "device use node001; biossettings; set boot mode 'legacy bios mode';  
commit"
```

```
cmsh -c "device use node001; biossettings; set boot mode 'uefi mode'; commit"
```

```
# Configure Node Interleaving
```

```
cmsh -c "device use node001; biossettings; set node interleaving enabled;  
commit"
```

```
cmsh -c "device use node001; biossettings; set node interleaving disabled;  
commit"
```

```
# Clear BIOS parameter (set to default)
```

```
cmsh -c "device use node001; biossettings; clear 'boot mode'; commit"
```

BIOS Configuration Application (3-Step Process)

Shell

```
# Step 1: Configure BIOS settings in CMDaemon database
```

```
cmsh -c "device use node001; biossettings; set boot mode 'legacy bios mode';  
commit"
```

```
# Step 2: Apply configuration to pending BIOS state
```

```
cmsh -c "device use node001; bios apply"
```

```
# Step 3: Activate by rebooting (makes pending settings live)
```

```
cmsh -c "device use node001; reboot"
```

```
# Verify final state after reboot
```

```
cmsh -c "device use node001; biossettings; status | grep 'Boot Mode'"
```

Firmware Management via CMDaemon

Firmware Management Mode Setup

Shell

Set firmware management mode for BMC settings

```
cmsh -c "device use node001; bmcsettings; set firmwaremanagemode<TAB><TAB>"
```

```
cmsh -c "device use node001; bmcsettings; set firmwaremanagemode auto; commit"
```

```
cmsh -c "device use node001; bmcsettings; set firmwaremanagemode h100; commit"
```

DGX H100

```
cmsh -c "device use node001; bmcsettings; set firmwaremanagemode ilo; commit"
```

HPE iLO

Set at category level

```
cmsh -c "category use dgx-h100; bmcsettings; set firmwaremanagemode h100; commit"
```

Set at partition level (global)

```
cmsh -c "partition use base; bmcsettings; set firmwaremanagemode h100; commit"
```

Basic Firmware Operations

Shell

List available firmware packages on head node

```
cmsh -c "device firmware info"
```

Show firmware status on specific node

```
cmsh -c "device firmware status -n node001"
```

Show firmware status with component details

```
cmsh -c "device firmware status -n node001 | grep -E '(Component|HGX_FW)'"
```

HPE iLO Firmware Management

Shell

Upload firmware to iLO (HPE specific)

```
cmsh -c "device firmware upload iLO5-2.42 -n node001"
```

List uploaded firmware on iLO node

```
cmsh -c "device firmware list -n node001"
```

```
# Flash firmware on iLO
cmsh -c "device firmware flash iLO5-2.42 -n node001"

# Monitor flash progress
cmsh -c "device firmware status -n node001"

# Remove firmware from iLO storage
cmsh -c "device firmware remove iLO5-2.42 -n node001"
```

DGX H100 Firmware Management

```
Shell

# Flash DGX firmware package
cmsh -c "device firmware flash nvfw_dgx-hgx-h100x8_0002_230705.1.1.fwpkg -n node001"

# Monitor DGX firmware flash progress
cmsh -c "device firmware status -n node001 | grep nvfw"

# Check specific components during flashing
cmsh -c "device firmware status -n node001 | grep 'HGX_FW_GPU'"
cmsh -c "device firmware status -n node001 | grep 'HGX_FW_NVSwitch'"

# View firmware version transitions
cmsh -c "device firmware status -n node001 | head -2 | cut -b66-133; device firmware status -n node001 | grep nvfw | cut -b66-133"
```

Advanced Firmware Options

```
Shell

# List available firmware targets (DGX)
cmsh -c "device firmware flash --targets list -v -n node001"

# Force firmware downgrade (DGX)
cmsh -c "device firmware flash FIRMWARE_PACKAGE --force -n node001"

# Dry run firmware update (DGX)
cmsh -c "device firmware flash FIRMWARE_PACKAGE --dry-run -n node001"

# Target specific components (DGX)
```

```
cmsh -c "device firmware flash FIRMWARE_PACKAGE --targets
HGX_FW_GPU_SXM_1,HGX_FW_GPU_SXM_2 -n node001"
```

Hardware Profile Verification

Hardware Profile Setup

```
Shell
# Create category for new hardware batch
cmsh -c "category add newbunch; commit"

# Mass assign nodes to category
for i in {129..255}; do
    cmsh -c "device; set node00$i category newbunch; commit"
done

# More efficient mass assignment
(echo device;
for i in {129..255}; do
    echo "set node00$i category newbunch"
done
echo "commit") | cmsh
```

Hardware Profile Creation and Monitoring

```
Shell
# Save hardware profile for reference node
/cm/local/apps/cmd/scripts/healthchecks/node-hardware-profile -n node129 -s
newbunch

# Enable hardware profile monitoring
cmsh -c "monitoring setup use hardware-profile"
cmsh -c "monitoring setup use hardware-profile; set interval 600; set disabled
no; commit"

# Configure category-specific monitoring
cmsh -c "monitoring setup use hardware-profile; nodeexecutionfilters"
cmsh -c "monitoring setup use hardware-profile; nodeexecutionfilters; add
category filterhwp"
```

```
cmsh -c "monitoring setup use hardware-profile; nodeexecutionfilters use
filterhwp; set categories newbunch; commit"
```

Advanced BIOS and Firmware Patterns

Template Management

```
Shell
# List available BIOS templates
ls -la /cm/local/apps/cm-bios-tools/templates/

# View template content
cat /cm/local/apps/cm-bios-tools/templates/dell_15g-intel.json
cat /cm/local/apps/cm-bios-tools/templates/hpe_dl380g10.json

# Use cm-bios-manage utility directly
/cm/local/apps/cm-bios-tools/bin/cm-bios-manage -t # List models
/cm/local/apps/cm-bios-tools/bin/cm-bios-manage -p hpe_dl380 # List profiles
/cm/local/apps/cm-bios-tools/bin/cm-bios-manage -T hpe_dl380 # Show template
```

Firmware Package Management

```
Shell
# Firmware package location
ls -la /cm/local/apps/cmd/etc/htdocs/bios/firmware/
ls -la /cm/local/apps/cmd/etc/htdocs/bios/firmware/h100/

# Copy firmware packages to correct location
cp nvfw_DGX-H100_*.fwpkg /cm/local/apps/cmd/etc/htdocs/bios/firmware/h100/
chmod 644 /cm/local/apps/cmd/etc/htdocs/bios/firmware/h100/*.fwpkg
```

BMC Network Configuration for Firmware Management

```
Shell
# Set up BMC network for Redfish
cmsh -c "network add bmcnet"
```

```

cmsh -c "network use bmcnet; set baseaddress 10.148.0.0; set domainname
bmc.cluster; commit"

# Configure BMC settings at partition level
cmsh -c "partition use base; bmcsettings"
cmsh -c "partition use base; bmcsettings; set username admin; set userid 0;
commit"
cmsh -c "partition use base; bmcsettings; set password PASSWORD; commit"
cmsh -c "partition use base; bmcsettings; set firmwaremanagemode h100; commit"

# Add BMC interface to node
cmsh -c "device use node001; interfaces; add bmc ipmi0"
cmsh -c "device use node001; interfaces use ipmi0; set network bmcnet; set ip
10.148.0.1; commit"

```

DGX H100 Specific Operations

DGX Power Cycling Requirements

```

Shell

# Note: DGX H100 requires AC power cycle for firmware activation
# Regular DC power reset is NOT sufficient

# Check for pending firmware requiring AC power cycle
cmsh -c "device firmware status -n node001 | grep 'AC power cycle'"

# Firmware status after flashing (before AC power cycle)
cmsh -c "device firmware status -n node001 | grep -E '(pending|AC power
cycle)'"

# Verify successful activation after AC power cycle
cmsh -c "device firmware status -n node001 | grep -E '(completed|success:
activated)'"

```

DGX Component-Specific Updates

```

Shell

# GPU tray firmware
cmsh -c "device firmware flash nvfw_dgx-hgx-h100x8_*.fwpkg -n node001"

```



```
# Chassis firmware
cmsh -c "device firmware flash nvfw_dgx-h100_*.fwpkg -n node001"

# Monitor GPU-specific components
cmsh -c "device firmware status -n node001 | grep 'HGX_FW_GPU_SXM'"
cmsh -c "device firmware status -n node001 | grep 'HGX_FW_NVSwitch'"
cmsh -c "device firmware status -n node001 | grep 'HGX_FW_PCIERetimer'"
```

Troubleshooting and Monitoring Commands

BIOS Troubleshooting

```
Shell

# Check for BIOS model errors
cmsh -c "device use node001; biossettings; status" | grep -i error

# Verify BIOS configuration states
cmsh -c "device use node001; biossettings; status | head -5"

# Clear problematic BIOS settings
cmsh -c "device use node001; biossettings; clear 'parameter name'; commit"

# Reset all BIOS to defaults
cmsh -c "device use node001; biossettings; clear; commit"
```

Firmware Troubleshooting

```
Shell

# Check for failed firmware operations
cmsh -c "device firmware status -n node001 | grep -E '(exception|failed)'"

# Monitor firmware progress
watch "cmsh -c 'device firmware status -n node001 | grep flashing'"

# Check firmware operation results
cmsh -c "device firmware status -n node001 | grep -E '(Result|Error)'"

# View detailed firmware status
```

```
cmsh -c "device firmware status -n node001" | grep -v current
```

Mass Operations Monitoring

Shell

Check BIOS compliance across category

```
cmsh -c "device foreach -c CATEGORY (bios check)" | grep different
```

Monitor firmware status across multiple nodes

```
cmsh -c "device foreach -n node001..node010 (firmware status)" | grep -E  
"(flashing|pending|exception)"
```

Generate compliance reports

```
cmsh -c "device foreach -c production (bios check)" >  
bios-compliance-report.txt
```

```
cmsh -c "device foreach -c production (firmware status)" >  
firmware-status-report.txt
```

Quick Reference Patterns

Standard BIOS Configuration Workflow

Shell

1. Set model

```
cmsh -c "device use NODE; biossettings; set model MODEL_NAME; commit"
```

2. Configure parameters

```
cmsh -c "device use NODE; biossettings; set PARAMETER VALUE; commit"
```

3. Apply configuration

```
cmsh -c "device use NODE; bios apply"
```

4. Activate with reboot

```
cmsh -c "device use NODE; reboot"
```

Standard Firmware Update Workflow

Shell

1. Set firmware management mode

```
cmsh -c "device use NODE; bmcsettings; set firmwaremanagemode MODE; commit"
```

2. Flash firmware

```
cmsh -c "device firmware flash FIRMWARE_PACKAGE -n NODE"
```

3. Monitor progress

```
cmsh -c "device firmware status -n NODE"
```

4. AC power cycle (for DGX H100)

Physical AC power cycle required

5. Verify activation

```
cmsh -c "device firmware status -n NODE | grep 'success: activated'"
```

Emergency Recovery

Shell

Clear all BIOS settings to defaults

```
cmsh -c "device use node001; biossettings; clear; commit"
```

Check for stuck firmware operations

```
cmsh -c "device firmware status -n node001 | grep -E '(flashing|pending)'"
```

Reset BIOS model configuration

```
cmsh -c "device use node001; biossettings; clear model; commit"
```

Common One-Liners

Shell

Mass BIOS compliance check

```
cmsh -c "device foreach -c CATEGORY (bios check)" | grep different
```

Quick firmware status overview

```
cmsh -c "device foreach (firmware status)" | grep -E  
"(node|flashing|pending|exception)"
```

Set boot mode for entire category

```
cmsh -c "category use CATEGORY; biossettings; set boot mode 'uefi mode';  
commit"
```

Monitor DGX firmware progress

```
cmsh -c "device firmware status -n node001" | grep nvfw
```